TNM089 – Imaging Technology
Lab 2 – Digital camera image processing chain

References

The basics:
Lecture 2: Digital camera image processing chain + Demosacing
Lecture 3: Automatic White Balancing

The Details:

- Digital camera image processing chain:
  Lukac. "*Single-Sensor Imaging: Methods and Applications for Digital Cameras*", Chapter 3
- Demosaicing:
  Losson et. el. "*Comparison of color demosaicing methods*"
- Automatic White Balance:
  Lukac. "*Single-Sensor Imaging Methods and Applications for Digital Cameras*", Chapter 10

Task 1 – Image acquisition

Load the mat-file "Lab2_data".
"MS_Image" is a multispectral image of size 1040 x 1392 x 31, i.e. sampled at 31 spectral bands. "Camera" is a 3 x 31 matrix containing the spectral sensitivity RGB-functions for a real camera. "wl" is a vector containing the wavelengths (wl = 420:10:720 nm). If you want, you can plot the camera sensitivity functions for each wavelength to investigate the sensitivity curves for R, G and B.

Generate a raw sensor RGB-image by computing the camera response (using the camera RGB-sensitivity functions) for the multispectral image. For *each pixel* in the image, the sensor response is given by:

a_n = sum(Mn.*I)

where a_n is the sensor response for channel n (n = 1,2,3 for an RGB-image) and I is the spectrum recorded at that pixel in the multispectral image. Make sure to get the matrix-dimensions right!

The result should be a 1040 x 1032 x 3 RGB-image. Display the image to verify that it's correct. This RGB-image is linear and may appear to have less contrast and dynamic range compared to an RGB-image rendered for viewing. This image represents the reference image that's projected directly on the sensor, before image data is discarded by the color filter array and sensor noise is added.

Task 2 – CFA-simulation and Demosaicing

From the reference RGB-image: create a color filter array (CFA) image, using the conventional Beyer-pattern. The result is an image containing color data for 50% of the G-pixels and 25% for the R- and B-pixels respectively.

From the CFA-image: perform demosaicing to compute the full RGB-image:

1. Use bilinear interpolation for each channel independently.
2. Make use of the spectral correlation assumption to estimate the R- and B-channels (Hue constancy or Color difference constancy, Le2 #44-45).
3. Make use of the spatial correlation assumption and add edge directed interpolation for the G-channel (Le2 #47).

Evaluate the different demosaicing methods by comparing the result to the reference RGB-image. Compute the mean square error for each method to investigate how close the result is to the reference. See if you can find regions in the resulting images where the difference is visible.

Optional: Calculate the S-CIELAB color difference between the original and demosaiced images, using appropriate screen resolutions to determine the viewing distance at which the difference becomes visible.

Task 3 – Automatic White Balance

The spectral data in "MS_Image" is the product of the light source (unknown) and the reflecting surfaces in the scene. Automatic White Balancing (AWB) aims to estimate the light source in a scene to adjust the colors so that the colors look neutral.

Use the demosaiced RGB image from Task 2 to:

1. Implement AWB using the *Gray world* assumption to correct the RGB image (Le3 #43-44)
2. Implement AWB using the *White patch* method to correct the RGB image (Le3 #45-46)
3. This image contains a standard reference white sample (a highly reflective barium sulfate surface) that can be used as a reference. Use the physical white reference to perform AWB. You can use the 100 x 100 sub-image: RGB(750:850, 1200:1300,:), where all pixels belong to the reference white.

Compare the results of the different methods. Do they differ? Which one do you prefer? Which is the most correct one?

Optional: Implement AWB using Iterative white balancing and evaluate the results.

Task 4 – Image space rendering

The RGB-images you have worked with and adjusted are still in a linear color space. The last steps aim to adjust the image for display, making it "look good" for an observer. Experiment how you can adjust the image to look more pleasing on screen, using e.g.:

- Gamma correction to optimize for non-linear displays (typically using γ=1/2.4)
- Tone scale corrections to enhance image contrast (contrast stretching and/or S-shaped transfer function)
- Edge enhancement (e.g. using unsharp masking)

Explain the operations you choose and motivate your choice of parameters. Does the order matter? When you're satisfied with the result, convert the image to UINT8. This is the final output 8-bit image, prepared for display. Compared to the original reference image (in Task 1): which one do you prefer?

Hints:

- You have previously implemented contrast stretching, gamma correction and unsharp masking in TNM087, Lab 1 & 2.
- A simple method to increase image contrast is to use a sigmoidal S-Curve: `Im2 = 1./(1 + exp(-intensity*(Im - 0.5)));` where `intensity` determines the amount of contrast added (i.e. how steep the S-curve is).